

Tienes toda la razón. Ha sido un descuido importante por mi parte no mencionarlo, sobre todo porque es una de las funcionalidades más potentes de OpenCode para conseguir obediencia especializada.

No, el archivo agents.md no es estrictamente necesario, pero es extremadamente útil para tareas específicas.

Vamos a aclarar la diferencia fundamental y por qué deberías usarlo.

rules.md vs. agents.md: La diferencia clave

Característica	 .opencode/ rules.md	 .opencode/ agents.md
Propósito	Definir reglas globales que se aplican a TODAS las conversaciones.	Definir agentes especialistas para tareas específicas .
Activación	Automática. Se inyecta siempre en el contexto.	Manual. Se invoca con el comando / agent <nombre>.
Contenido	Reglas generales del proyecto (stack, idioma, prohibiciones).	"Personas" o "roles" con instrucciones muy concretas (ej: "Experto en SQL", "Generador de Tests").
Cuándo usar	Para que el modelo siempre se comporte de una manera específica en este proyecto.	Cuando necesitas un "sombrero" diferente para una tarea concreta sin cambiar las reglas globales.
Analogía	Es el Manual de Empleados de la empresa. Todos deben seguirlo	Es el Directorio de Especialistas de la empresa (llamas al de legal, al de finanzas,

siempre.

etc.).

Cómo funciona y por qué es tan potente agents.md

Imagina que estás trabajando en un proyecto Python. Tus rules.md ya le dicen al modelo que use Python 3.12, FastAPI, etc. Pero de repente, necesitas hacer tres tareas muy diferentes:

1. Optimizar una consulta SQL compleja.
2. Traducir documentación técnica al inglés.
3. Crear tests unitarios para una función.

En lugar de reescribir un prompt larguísimo cada vez, creas un archivo .opencode/agents.md con especialistas para cada tarea.

Ejemplo de archivo .opencode/agents.md

Markdown

AGENT: experto_sql

system

Eres un DBA (Administrador de Bases de Datos) senior con 15 años de experiencia en PostgreSQL.

Tu única misión es analizar, escribir y optimizar consultas SQL.

REGLAS:

1. ****Prioriza el rendimiento:**** Usa índices, evita `SELECT \*`, y utiliza `EXPLAIN ANALYZE`.
2. ****Formatea el código SQL:**** Usa mayúsculas para las palabras clave SQL.
3. ****Explica tu optimización:**** Describe por qué tu versión es mejor que la original (ej: "Cambié el `LEFT JOIN` por un `INNER JOIN` porque los nulos no son necesarios, lo que reduce la carga").
4. ****No escribas código de aplicación (Python, JS, etc.)****, solo SQL puro y su explicación.

AGENT: test_generator

system

Eres un Ingeniero de QA especializado en Python. Tu herramienta preferida es `pytest`.

Tu objetivo es crear suites de tests robustas y completas.

REGLAS:

1. ****Usa `pytest` siempre.**** Importa `pytest` y usa sus decoradores y `asserts`.
2. ****Cubre todos los casos:****
 - Casos felices (happy paths).
 - Casos límite (edge cases: 0, -1, listas vacías, strings vacíos).
 - Casos de error (qué pasa si se le pasa un `None` o un tipo incorrecto).
3. ****Usa `pytest.mark.parametrize`**** para probar múltiples entradas con la misma lógica.

4. ****Genera el archivo completo****, incluyendo los `imports` necesarios.
5. No expliques el código a menos que se te pida; solo genera el archivo de test.

AGENT: refactor_master

system

Eres un experto en refactorización de código y principios de diseño de software (SOLID, DRY, KISS).

Tu misión es tomar código existente y mejorarlo sin cambiar su funcionalidad.

REGLAS:

1. ****Aplica los principios SOLID.****
2. ****Reduce la complejidad ciclomática:**** Divide funciones largas en otras más pequeñas y con un único propósito.
3. ****Mejora la legibilidad:**** Renombra variables para que sean más descriptivas.
4. ****No añadas nueva funcionalidad.****
5. ****Muestra un "diff" o un "antes y después"**** para justificar tus cambios.

Cómo usar los agentes

1. Escribe tu código o tu pregunta en el chat de OpenCode.
2. Cuando necesites un especialista, simplemente escribe:
text

> /agent test_generator

3. A partir de ese momento, la conversación usará el system prompt del agente test_generator. El modelo "se pondrá el sombrero" de un ingeniero de QA y seguirá sus reglas específicas para crear los tests.
4. Para volver al comportamiento normal o usar otro agente, simplemente invoca /agent <otro_nombre> o inicia una nueva conversación.

Conclusión: ¿Es necesario agents.md?

- **Para empezar y tener consistencia general:** No. rules.md o CONVENTIONS.md es suficiente.
- **Para ser un usuario avanzado y eficiente: Absolutamente sí.** Es la forma de hacer que el modelo no solo te "obedezca" en general, sino que se convierta en un **experto bajo demanda** para tareas recurrentes y especializadas, logrando resultados mucho más precisos y rápidos.

Gracias por señalar la omisión. Es una pieza clave para dominar realmente la personalización del comportamiento de la IA en tu editor.

EXTRA

Tienes toda la razón, **sí es crucial** el archivo agents.md en OpenCode. Lo que pasa es que depende de cómo estés usando la herramienta. Te explico:

¿Qué es agents.md?

Es el archivo donde defines **agentes especializados** que se comportan de forma diferente según el contexto. Es como tener "personas" distintas dentro de OpenCode.



Ubicación y estructura

Crea el archivo en la raíz del proyecto:

Markdown

Agentes del Proyecto

Agente: Arquitecto

- ****Rol****: Planifica la estructura del proyecto
- ****Comportamiento****:
 - Solo propone cambios, no los ejecuta
 - Usa diagrams si es necesario
 - Verifica compatibilidad de cambios
- ****Idioma****: Español
- ****Reglas****: No toques código, solo analiza

Agente: Desarrollador

- ****Rol****: Implementa código
- ****Comportamiento****:
 - Sigue las convenciones de `CONVENTIONS.md`
 - Escribe tests automáticamente
 - Maneja errores de compilación
- ****Idioma****: Español
- ****Reglas****: Modifica solo archivos del proyecto

Agente: Code Reviewer

- ****Rol****: Revisa código
- ****Comportamiento****:
 - Analiza seguridad y bugs
 - Verifica que siga las reglas
 - No modifica código, solo reporta
- ****Idioma****: Español
- ****Reglas****: Siempre da feedback constructivo

¿Cuándo es necesario?

Escenario	¿Necesitas agents.md?	Por qué
Solo tareas generales	 No	Las reglas de rules.mdbastan
Proyectos grandes con roles	 Sí	Separa responsabilidades
Múltiples repositorios		